Since 1983

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Department of Computer Engineering and Technology

MINI PROJECT REPORT

Computer Networks Laboratory

(CSE20110)

Laboratory Continuous Assessment 3

PEER TO PEER FILE DISTRIBUTION SYSTEM

Submitted by

Sr. No.	Student Name	PRN
1	Rachel Tom	1262240226
2	Manasi Sharma	1262240248
3	Aaron Johnson	1262240581
4	Aaron Antao	1262240595

Group No.5 Panel K Batch 1

Third Year B.Tech. Computer Engineering and Technology

Under the guidance of

Dr Uma Pujeri

Academic Year 2026–2027

Submission Date 19 October 2026

Since 1983

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

MIT-WPU

॥ विश्वशान्तिर्ध्रुवं ध्रुवा ॥

Department of Computer Engineering and Technology

CERTIFICATE

This is to certify that the mini-project entitled "Peer to Peer File Distribution System" has been carried out by the following students of Third Year B.Tech. Computer Science and Engineering. Panel, batch, group number and student details are to be completed by the group before submission.

Student Name	**PRN**
Rachel Tom	1262240226
Manasi Sharma	1262240248
Aaron Johnson	1262240581
Aaron Antao	1262240595

The report is submitted towards the mini-project component of Laboratory Continuous Assessment 3 for the Computer Networks Laboratory course.

Faculty Signature _____

Faculty Name To be completed

Project Guide and Course Faculty

Date _____ Place: Pune

Acknowledgement

We thank our course faculty and the Department of Computer Engineering and Technology for the guidance and laboratory environment used for this mini-project. We also acknowledge the Python standard-library documentation and the project test environment, which were used to design, implement and verify the socket-based prototype.

Group No. 5

Abstract

This project implements a peer-to-peer file distribution prototype in which a file is divided into fixed-size chunks and distributed directly between peers. A lightweight tracker maintains chunk availability and the latest version of each file; it does not relay file contents. Each peer runs a TCP server, requests missing chunks from available owners, verifies each received chunk using SHA-256, stores it locally and immediately announces it as available for upload. When the source file is updated, the seeder publishes a new versioned manifest, the tracker notifies every peer, and each peer downloads the chunks it does not already hold from the other peers before rebuilding the new version. The implementation is written in Python using only the standard library and uses newline-delimited JSON control messages over TCP. The project contains a chunking utility, tracker, peer application and shared protocol helper. In a local test, a peer downloaded version 1 of DSL.pdf (4 chunks), was notified when version 2 (26 chunks, 1,639,852 bytes) was published, downloaded the new version and rebuilt it as DSA_areceived.pdf with a SHA-256 digest identical to the source. The design demonstrates decentralised chunk sharing, integrity checking, resumable local storage and version updates. It is suitable as a compact laboratory prototype and can be extended with parallel downloads, authenticated peers, signed manifests and a distributed tracker.

Keywords: peer-to-peer networking, file chunking, TCP sockets, SHA-256 integrity verification, version update notification

Table of Contents

1 Introduction and Project Objectives 6
1.1 Introduction and Background 6
1.2 Problem Statement 6
1.3 Scope 6
1.4 Objectives 6
2 Proposed Design and Requirements 7
2.1 Architecture and Network Topology 7
2.2 Tools and System Requirements 8
2.3 Addressing and Protocol Selection 8
3 Implementation and Technical Working 10
3.1 Methodology 10
3.2 Configuration or Code Implementation 10
3.3 Protocols and Technical Working 10
3.4 Update Notification and Consistency 11
3.5 Execution Procedure 12
4 Testing and Results 13
4.1 Test Plan 13
4.2 Results and Execution Evidence 14
4.3 Objective Achievement 15
5 Discussion and Conclusion 16
5.1 Discussion and Comparative Analysis 16
5.2 Advantages and Applications 16
5.3 Limitations 16
5.4 Challenges and Troubleshooting 16
5.5 Future Scope 17
5.6 Conclusion 17
References 19
AI Usage Declaration 19
Appendices 20
Appendix A Code and Configuration 20
Appendix B Additional Evidence 20
Appendix C Individual Contributions 20

1 Introduction and Project Objectives

1.1 Introduction and Background

Peer-to-peer (P2P) distribution avoids sending every copy of a file from one central server. Instead, participants that possess verified pieces contribute upload capacity to other participants. BitTorrent popularised this piece-based model: a torrent metainfo file identifies the content, peers exchange availability information, and integrity hashes identify valid pieces [1]. This project applies the same core ideas in a deliberately small laboratory prototype. TCP provides ordered, reliable byte delivery between the tracker and peers [2]. SHA-256 is used to check every stored chunk before it can be shared, following the secure-hash construction specified by NIST [3].

1.2 Problem Statement

A single file server becomes a bottleneck when many users request the same large file and represents a single point of failure. The problem is to distribute a file while allowing peers to obtain chunks from available nodes and to begin contributing data before their own complete copy is assembled. In addition, when the original file changes, every node holding a copy must be told and must obtain the updated copy, without the source sending the whole file to each node again.

1.3 Scope

The system is intended for a controlled LAN or laboratory demonstration using Python 3.10 or later. It assumes peers can reach the tracker and one another over TCP, share a trusted initial manifest out of band (later versions are obtained from the tracker) and use stable host addresses during a session. It is a prototype: NAT traversal, encryption, user authentication, incentives and a distributed tracker are outside the present scope.

1.4 Objectives

The objectives and corresponding verification methods are listed in Table 1.1.

Objective ID	Objective	Verification Method
O1	Split a file and create a SHA-256 manifest	make_manifest.py output and manifest inspection

Objective ID	Objective	Verification Method
O2	Retrieve, verify and seed chunks through TCP peers	peer download log and chunk folder
O3	Reconstruct the original file correctly	SHA-256 equality of source and output
O4	Notify every node when the source file is updated and deliver the updated copy through peers	update_available log, re-downloaded chunk count and SHA-256 of the new output

2 Proposed Design and Requirements

2.1 Architecture and Network Topology

Figure 2.1 shows the topology. The tracker indexes which peer owns each chunk, stores the latest manifest of each file and returns this information in response to lookup and announce requests. It never carries file bytes. Peer A, the seeder, initially owns all chunks and publishes each new version of the file. Peers B and C request chunks over direct peer-to-peer TCP connections. After verification, each receiving peer writes the chunk to its local store and announces it, allowing it to seed subsequent requests. When a new version is published, the tracker sends update_available to every peer, and the changed chunks spread through the peers in the same way.

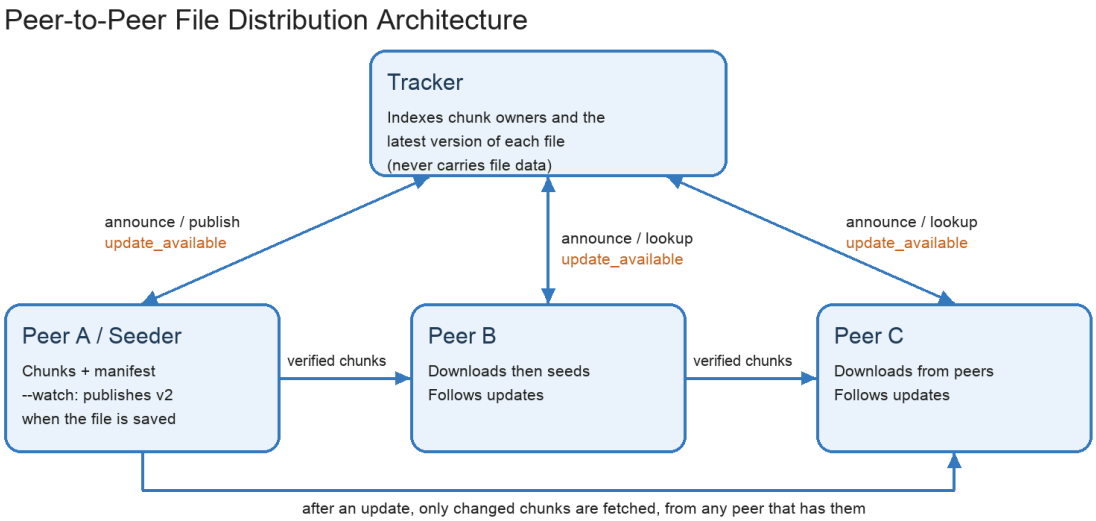


Figure 2.1 Peer-to-peer distribution architecture

2.2 Tools and System Requirements

The implementation uses the Python standard library only. The verified execution was performed locally on macOS using Python 3.10 or later. No third-party packages are required; the relevant tools are listed in Table 2.1.

Tool or Resource	Version or Specification	Purpose
Python	3.10 or later	TCP sockets, JSON, SHA-256 and file I/O
Operating system	macOS local environment	Local multi-terminal execution
Wireshark or tcpdump	Optional	Observe newline-delimited TCP protocol messages

2.3 Addressing and Protocol Selection

The system uses TCP sockets on loopback addresses for the demonstrated local run. The tracker listens at 127.0.0.1:9000. Seed and download peers can listen at 127.0.0.1:9101 and 127.0.0.1:9102 respectively. Each request or response is one compact JSON object terminated by a newline. Chunk payloads are Base64-encoded in peer responses.

Device / Interface	IP / Prefix	Gateway	Role
Tracker	127.0.0.1:9000	Not applicable	Chunk availability index, latest version and update notifications
Seeder peer	127.0.0.1:9101	Not applicable	Initial complete chunk owner and publisher of new versions
Download peer	127.0.0.1:9102	Not applicable	Downloader and subsequent seeder

TCP was selected because a chunk must arrive in order and without loss at the application layer. Newline-delimited JSON keeps control messages readable in a packet capture. The tracker stores file ID, chunk index, endpoint and last-seen time. It also stores the newest manifest for each file name and the peers that follow that file, so that it can push update notifications. A 90-second liveness interval removes stale owners. SHA-256 verification prevents a peer from retaining or advertising corrupted chunk data.

3 Implementation and Technical Working

3.1 Methodology

First, make_manifest.py reads a source file in fixed-size blocks, writes chunk files and records each chunk digest in manifest.json. The original seeder announces its complete set to tracker.py. A downloading peer looks up owners, obtains a missing chunk with get_chunk, decodes and verifies the data, writes it to its chunk folder, and re-announces availability. Once all indices are present, the peer reconstructs the output by concatenating chunks in manifest order. For updates, the seeder is started with --watch. When the source file has been saved and has stayed unchanged for two seconds, the seeder splits it again, builds manifest version N+1 and publishes it to the tracker. The tracker records it as the latest version and pushes update_available to every live peer. Each peer fetches the new manifest, keeps the chunks whose hash is unchanged, downloads the remaining chunks from other peers using the same lookup and get_chunk steps, and replaces its output file with the new version.

3.2 Configuration or Code Implementation

protocol.py serialises one JSON message per TCP connection. tracker.py handles announce, lookup, publish and get_manifest actions through a threaded TCP server and pushes update_available messages to peers. peer.py contains ChunkStore, which stores each chunk under its SHA-256 digest and accepts only data that matches it; a Peer class that serves get_chunk, receives update notifications, drives the download loop and rebuilds the output; and the --watch option that publishes new versions from the seeder. make_manifest.py creates the hash-named chunks and the versioned manifest, which records the file name, version, size, whole-file SHA-256, chunk size and per-chunk digests. The project has no third-party runtime dependency.

3.3 Protocols and Technical Working

The tracker accepts announce messages containing the file name, file ID, host, port and owned chunk list, then records a timestamp for each ownership relation; its reply includes the latest published version of that file. A lookup returns an index-to-owner map. The downloader chooses a candidate owner, opens a peer TCP connection and sends get_chunk with the chunk's SHA-256 digest. The receiving peer returns Base64 content or an error. Failed connections, invalid responses and hash mismatches are caught; the downloader logs the failure and tries another owner on the next pass. A completed peer remains running and re-announces its chunks every 25 seconds. Three messages support updates: publish (seeder to tracker, carrying the new

manifest), get_manifest (peer to tracker, returning the newest manifest) and update_available (tracker to peer, carrying the file name, version and file ID). Table 3.1 lists all messages.

Message	Direction and fields	Purpose
announce	Peer to tracker: name, file_id, host, port, chunks	Register owned chunks; reply carries the latest version
lookup	Peer to tracker: file_id	Return the owners of each chunk
get_chunk	Peer to peer: sha256	Return the chunk data in Base64
publish	Seeder to tracker: manifest	Record a new version and notify peers
get_manifest	Peer to tracker: name	Return the newest manifest
update_available	Tracker to peer: name, version, file_id	Tell a peer that a newer version exists

3.4 Update Notification and Consistency

A notification alone is not enough: the update must not corrupt any node's copy. Chunks are stored as <sha256>.bin, so two versions never overwrite each other and unchanged chunks are shared between versions automatically. Every received chunk is checked against the digest in the manifest of the version being built, so an old or damaged chunk cannot enter the new file. The manifest's file ID is a hash of its fields, so a damaged manifest is rejected, and the whole-file SHA-256 is checked after rebuilding. The output is written to a temporary file and then renamed over the old one, so it is always either the complete old version or the complete new one.

The seeder publishes only after the source file has stopped changing for two seconds, and waits again if the file changes while it is being split, so a half-saved file is never distributed. If a newer version is announced while a peer is still downloading, the peer abandons the older version and switches to the newest one, keeping any chunks it can reuse. A peer that misses the push notification, or starts later with an old manifest, learns of the newer version from the tracker's reply to its next announce.

3.5 Execution Procedure

Prerequisite: Python 3.10 or later. In Terminal 1 run python3 tracker.py. In Terminal 2 prepare the file using python3 make_manifest.py DSL.pdf --out shared. In Terminal 3 run python3 peer.py --manifest shared/manifest.json --chunks shared --listen 127.0.0.1:9101 --seed --watch DSL.pdf. In Terminal 4 run python3 peer.py --manifest shared/manifest.json --chunks peer-c --listen 127.0.0.1:9104 --output DSA_areceived.pdf. Then run shasum -a 256 DSL.pdf DSA_areceived.pdf; equal digests verify the reconstructed output. Figure 3.1 shows the manifest step and the final digest comparison: make_manifest.py split DSL.pdf into 26 chunks (the manifest was at version 3 because it had been generated in that folder before), and both files have the SHA-256 digest 494dd4ff95030e5a70b6d37aaab498c42d2adf67b115801cd8b108eaaee3395b. To test an update, save a new version of DSL.pdf while the peers are running: the downloader prints an update notice, fetches the chunks it does not hold and rebuilds DSA_areceived.pdf, after which the digests match again.

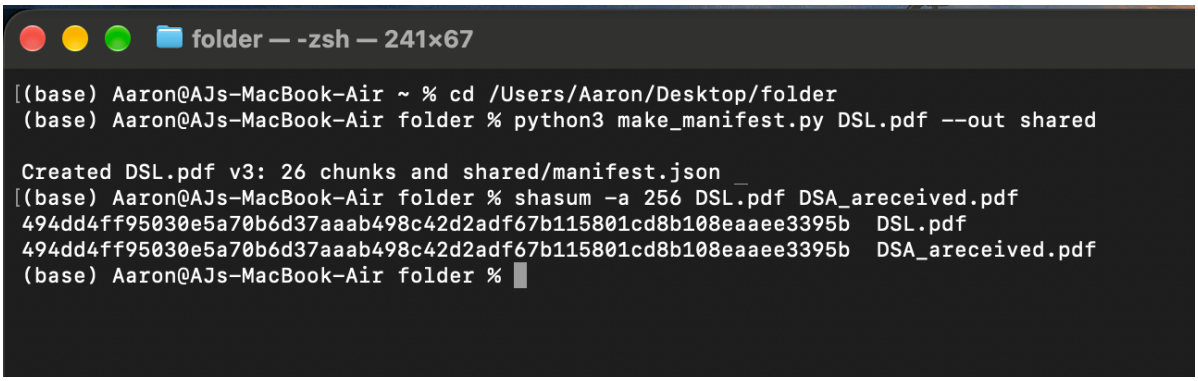


Figure 3.1 Local execution verification output

4 Testing and Results

4.1 Test Plan

Testing used the local project directory with the tracker, the seeder's shared directory and a downloading peer using the peer-c directory on 127.0.0.1:9104. Version 1 of DSL.pdf (197,278 bytes, 4 chunks) was distributed first; a new 1,639,852-byte DSL.pdf (26 chunks) was then published as version 2 while the peer was running. The success criteria were that the manifest is created, the peer acquires and seeds every chunk, the peer is notified of the new version and rebuilds it, and the output digest equals the source digest. The code also contains paths for unavailable chunks, failed connections and SHA-256 mismatch; these cases are logged and must not be stored as valid chunks. Tests T6 and T7 were run separately with generated files: a peer started with an outdated version 1 manifest, and a chunk file on the seeder overwritten with invalid data.

Test ID	Test and Input	Expected Result	Observed Result	Status
T1	Create manifest from DSL.pdf	Chunks and manifest are created	26 chunks and shared/manifest.json created (manifest v3)	Pass
T2	Download DSL.pdf v1 to DSA_areceived.pdf	Peer retrieves and seeds every chunk	Chunks 0 to 3 of v1 downloaded and seeded; v1 completed (197,278 bytes)	Pass
T3	Publish DSL.pdf v2 while the peer runs	Peer is notified of the new version	Peer printed “Update available: DSL.pdf v2 (you have v1)”	Pass
T4	Download the new version	Peer downloads every chunk it does not hold and rebuilds v2	26 of 26 chunks downloaded and seeded; v2 completed (1,639,852 bytes)	Pass
T5	SHA-256 of DSL.pdf vs DSA_areceived.pdf	Identical digest	Both digests: 494dd4ff...eaaee3395b	Pass
T6	Start a peer with the old v1 manifest after the update	Peer detects v2 and builds it	Peer stopped the v1 download and completed v2 with a matching digest	Pass

Test ID	Test and Input	Expected Result	Observed Result	Status
T7	Overwrite a chunk on the seeder with invalid data	Chunk is rejected and not saved	Receiver logged failed SHA-256 verification; no output was built from bad data	Pass

4.2 Results and Execution Evidence

The execution evidence in Figure 4.1 records the final digest comparison from the local test. make_manifest.py created 26 chunks and shared/manifest.json for DSL.pdf, and the two displayed SHA-256 values for DSL.pdf and the peer's rebuilt DSA_areceived.pdf are identical: 494dd4ff95030e5a70b6d37aaab498c42d2adf67b115801cd8b108eaaee3395b. This proves byte-for-byte reconstruction of the demonstrated file.

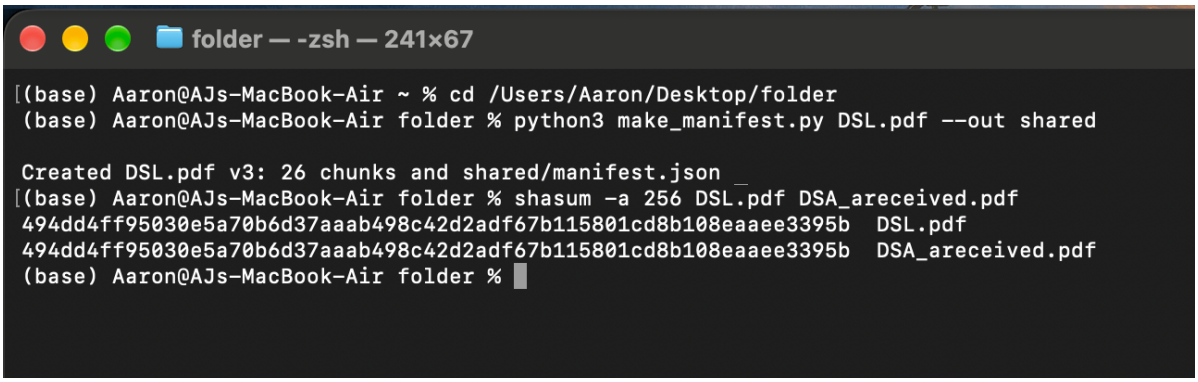


Figure 4.1 Matching SHA-256 verification result

Figure 4.2 records the update test from the downloading peer's terminal. The peer started with 0 of 4 chunks of DSL.pdf version 1, downloaded and seeded chunks 0 to 3, and completed DSA_areceived.pdf version 1 (197,278 bytes). When version 2 was published, the peer printed "Update available: DSL.pdf v2 (you have v1)". Because version 2 was a different, larger file, none of its chunks matched a chunk the peer already held, so the peer reported 26 of 26 chunks to download. It downloaded and seeded chunks 0 to 25 and completed DSA_areceived.pdf version 2 (1,639,852 bytes). The printed rates (33,643.1 KiB/s and 44,350.3 KiB/s) were measured on a single machine over the loopback interface and are not used as network throughput results.

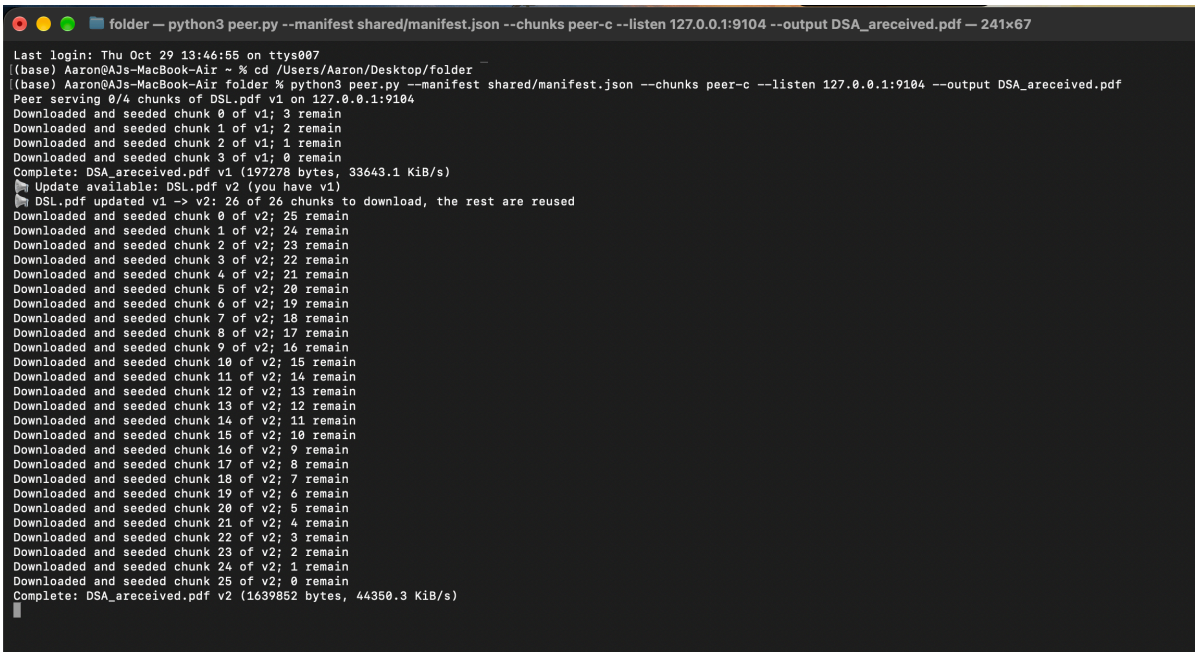


Figure 4.2 File update verification result

4.3 Objective Achievement

The verified local transfer demonstrates chunk generation, peer retrieval, integrity verification and output reconstruction. Immediate seeding is implemented and the download log reports “Downloaded and seeded chunk 0”; a multi-peer throughput measurement was not collected and remains future experimental work. Failure recovery is implemented through retries but was not measured as a separate live test in this report.

The update objective O4 is demonstrated by tests T3 to T6: the running peer was notified of the new version, downloaded every chunk it did not hold, and rebuilt a copy whose digest matched the new source, and a peer started with an outdated manifest still reached the latest version. Test T7 confirms that a corrupted chunk is rejected.

Objective ID	Evidence or Test ID	Outcome
O1	T1	Achieved
O2	T2, T4, T7	Achieved
O3	T5	Achieved
O4	T3, T4, T5, T6	Achieved

5 Discussion and Conclusion

5.1 Discussion and Comparative Analysis

Compared with a central-server model, the tracker here is not a data relay, so file-byte traffic can be shared among peers rather than concentrated at one endpoint. The completed local digest comparison verifies correctness but does not establish a speed improvement because no multi-peer timing data was collected. The main demonstrated result is functional distribution and integrity preservation, not a measured performance claim. For updates, a peer reuses every chunk whose hash is unchanged and downloads only the rest. In the DSL.pdf test version 2 was a different file, so all 26 chunks were new and had to be downloaded; when an edit changes only part of a file, only the affected chunks are transferred.

5.2 Advantages and Applications

The approach reduces reliance on one file source, permits partial peers to contribute upload capacity, and checks data before it is retained. It is relevant to software-image distribution in a lab, sharing large course material over a LAN, and content-distribution prototypes where peers can contribute spare upload bandwidth. The update mechanism is relevant to distributing new versions of software, course material or datasets, where only small parts of a large file change between versions.

5.3 Limitations

The tracker is centralised and is therefore still a control-plane single point of failure. Chunks are transferred as Base64 inside JSON, which is simple but inefficient for large payloads. Downloads are not parallelised, peers are not authenticated, and the shared manifest is assumed to be trusted. NAT traversal, TLS and malicious-peer resistance are not implemented. Any client that can reach the tracker can publish a new version, because manifests are not signed. Chunks have a fixed size, so inserting bytes near the start of a file shifts all later chunks and they must all be downloaded again. Chunks of old versions remain in each peer's folder until removed manually. If the tracker is unavailable, peers cannot discover owners or receive update notifications; the seeder publishes the current version again on its next announce once the tracker returns.

5.4 Challenges and Troubleshooting

During initial execution, the downloader reported FileNotFoundError for shared/manifest.json because the source file had not first been split. The correction was to run make_manifest.py

before starting the seeder or downloader. A later SHA-256 comparison between demo-file.txt and received.txt matched exactly, verifying the correct execution order and reconstruction. While adding update support, two design problems were found. The original file ID was a hash of the chunk list, so an edited file received an unrelated ID and peers could not tell that it was a new version of the same file; a stable file name and version number were added to the manifest. Chunks were also stored by index, so storing a new version could overwrite a chunk still needed for the old one; chunks are now stored under their SHA-256 digest.

Issue	Cause or Diagnosis	Solution and Verification
manifest.json not found	Source file had not been split	Ran make_manifest.py; manifest and chunk created
Potential corrupt chunk	Digest differs from manifest	ChunkStore rejects it; no invalid chunk is retained
Edited file not linked to old version	File ID derived only from chunk hashes	Added file name and version; tracker stores the latest manifest per name
Chunk overwrite between versions	Chunks stored by index	Chunks stored under their SHA-256 digest; tests T4 to T6 passed

5.5 Future Scope

Future work can request several chunks concurrently, select rarest chunks first, transmit raw binary frames instead of Base64 JSON, add TLS and peer authentication, implement tracker replication or a DHT, and collect throughput, latency and availability data across a larger Mininet topology. Further update work includes signing manifests so that only the owner can publish versions, content-defined chunking so that insertions do not shift every chunk, and removal of chunks no longer used by any version.

5.6 Conclusion

The project delivers a working socket-based P2P file distribution prototype. It splits a source into verified chunks, records ownership through a tracker, transfers chunks directly between peers and reconstructs the original file. The demonstrated source and output digests were identical. The implementation meets the core functional objectives while clearly identifying

the performance and security work required for a production-grade system. It also keeps every node current: when the source file is updated, peers are notified, fetch the chunks they do not already hold from one another and rebuild the new version, and in testing the rebuilt copy matched the updated source.

References

[1] B. Cohen, “The BitTorrent Protocol Specification,” 2008. [Online]. Available: https://www.bittorrent.org/beps/bep_0003.html. Accessed: Oct. 19, 2026.

[2] J. Postel, “Transmission Control Protocol,” RFC 793, Sep. 1981. [Online]. Available: https://doi.org/10.17487/RFC0793. Accessed: Oct. 19, 2026.

[3] National Institute of Standards and Technology, “Secure Hash Standard,” FIPS PUB 180-4, Aug. 2015. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.180-4. Accessed: Oct. 19, 2026.

AI Usage Declaration

AI assistance was used for code scaffolding (including the file update feature), explanation and report drafting. The group is responsible for checking the code, commands and report content. The reported execution results are the DSL.pdf transfer, update and SHA-256 comparison shown in Figures 3.1, 4.1 and 4.2, and the missed-update and corrupted-chunk tests in Section 4; unmeasured performance results have not been claimed.

Declaration: We take responsibility for the accuracy of this report and understand the submitted implementation and results. Sources and assistance used have been acknowledged.

Student Name	**PRN**	**Signature**
Rachel Tom	1262240226	
Manasi Sharma	1262240248	
Aaron Johnson	1262240581	
Aaron Antao	1262240595	

Appendices

Appendix A Code and Configuration

Source files included with the project are: tracker.py (tracker server and update notifications), peer.py (peer server, downloader and update handling), make_manifest.py (file splitting and versioned manifest generation), protocol.py (newline-delimited JSON helpers), README.md (run instructions), PROJECT_OVERVIEW.md and REPORT.md (design summaries), and requirements.txt (no third-party packages). No credentials or secrets are used.

Appendix B Additional Evidence

Verification calculation: SHA-256(DSL.pdf) = SHA-256(DSA_areceived.pdf) = 494dd4ff95030e5a70b6d37aaab498c42d2adf67b115801cd8b108eaaee3395b. This equality confirms byte-for-byte equality for the demonstrated output.

Update verification: the downloading peer completed DSA_areceived.pdf version 1 (197,278 bytes, 4 chunks) and, after the update notice, version 2 (1,639,852 bytes, 26 chunks). The SHA-256 digest of the final output equals that of DSL.pdf, as shown above.

Appendix C Individual Contributions

Complete the contribution table with the actual work performed by each group member before submission.

Student Name and PRN	Work Completed	Evidence or Deliverable
Rachel Tom	Testing and execution verification	SHA-256 test result
Manasi Sharma	Chunking and integrity verification	make_manifest.py, peer.py
Aaron Johnson	Report and presentation	This report and slides
Aaron Antao	Tracker and protocol design	tracker.py, protocol.py